Code Coverage Report

Kort forklaring af JaCoCo-rapporten og hvordan den blev brugt til at designe unit tests.

JaCoCo overblik

Backend

Sessions

Backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
controller		95 %		79 %	37	220	19	676	0	128	0	10
dao		97 %		95 %	6	98	6	275	0	36	0	10
persistence.model		100 %		98 %	1	90	0	144	0	55	0	15
route		100 %		n/a	0	43	0	121	0	43	0	11
Total	160 of 4.866	96 %	44 of 378	88 %	44	451	25	1.216	0	262	0	46

Created with JaCoCo 0.8.12.202403310830

Headerne

- Element: pakke, klasse eller metode der måles på.
- Missed Instructions: bytecode-instruktioner som tests ikke ramte.
- Cov.: coverage-procent for den kolonne den står ved siden af.
- Missed Branches: if/else- og switch-grene som ikke blev testet.
- Missed / Cxty: manglende og samlet cyclomatic complexity, altså antal mulige kodeveje.
- Missed / Lines: kildekodelinjer som ikke blev ramt, samt samlet antal linjer.
- Missed / Methods: metoder som ikke blev kaldt af tests, samt samlet antal metoder.
- Missed / Classes: klasser som ikke blev ramt, samt samlet antal klasser.

Resultat

Rapporten viser 96 % instruction coverage og 88 % branch coverage samlet. Der mangler 0 metoder og 0 klasser, så alle metoder og klasser er ramt mindst én gang. persistence.model ligger meget højt med 100 % instructions og 98 % branches.

Eksempel: email-validering

```
96.         return email;
97.     }
98.
99.     public void setEmail(String email) {
100.        ModelValidation.requireTextLength(email, "Email", 6, 254);
101.        if (!email.contains("@")) {
102.            throw new IllegalArgumentException("Email must contain @.");
103.        }
104.        if (!email.contains(".")) {
105.            throw new IllegalArgumentException("Email must contain a dot.");
106.        }
107.        if (email.startsWith("@")) {
108.            throw new IllegalArgumentException("Email must not start with @.");
109.        }
110.        if (email.endsWith("@")) {
111.            throw new IllegalArgumentException("Email must not end with @.");
112.        }
113.        this.email = email;
114.    }
115.
116.    public Integer getPcNumber() {
117.        return pcNumber;
118.    }
119.
120.    public void setPcNumber(Integer pcNumber) {
```

De grønne linjer viser at setEmail blev kørt. Branch-markeringerne viser at begge sider af if-reglerne blev dækket: manglende @, manglende punktum, starter med @ og slutter med @.

Hvordan rapporten hjalp unit tests

```
            class="str">" ",
            class="str">" ",
            class="str">"A",
            class="str">"AA",
            class="str">"AAA",
            class="str">"AAAA",
            class="str">"AAAAA",
            class="str">"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
            class="str">"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
            class="str">"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
            class="str">"player.example.com", class="com">// White box value: missing @
            class="str">"player@",
            class="str">"player@example", class="com">// White box value: missing dot br
            class="str">"@example.com", class="com">// White box value: starts with @ br
            class="str">"player.example@" class="com">// White box value: ends with @ br
    })
    void emailShouldRejectInvalidLengths(String email) {
        class="com">// Given
        License subject = new License();

        class="com">// Then
        assertThatThrownBy(() -> subject.setEmail(email))
                .isInstanceOf(IllegalArgumentException.class);
    }
```

Coverage-rapporten gjorde testdesignet konkret. Hver branch i setEmail blev omsat til en testværdi: uden @, uden punktum, starter med @, slutter med @ og værdier ved længdegrænserne. Derfor blev testen parameterized og white-box-baseret, så den dækker både positive og negative cases kort og systematisk.